

Импорты

```
import os
import random
import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import Dataset, DataLoader
from torchvision import transforms
import torch.fft as fft
from PIL import Image
import matplotlib.pyplot as plt
import pandas as pd
from tqdm import tqdm
import gc

DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"устройство: {DEVICE}")
```

устройство: cpu

конфиг

```
n_workers = 0 #количество потоков для даталoadера
model_path = 'best_model_dual_domain.pth' #путь к фалу с весами
batch_size = 32
```

путь к тестовым данным

```
import kagglehub
path = kagglehub.competition_download('ml-intensive-yandex-academy-spring-20
#path = '/content/data/dataset'
```

```
Downloading to /content/data/ml-intensive-yandex-academy-spring-2026.archive.
100%|██████████| 2.49G/2.49G [00:32<00:00, 83.3MB/s]Extracting files...
```

```
test_img_path = f'{path}/test_images'
```

✓ Лучшая архитектура и её идея

Лучшая архитектура - Двухдоменная сеть состоящая из двух частей:

классическая двойная свёртка(VGG-архитектура) на изображении и аналогичная

свёртка на частотной карте, распознающая частотные артефакты, характерные для StyleGAN. После объединения и финальный свёрточный слой

частотные утилиты

```
def compute_log_magnitude_spectrum(image_tensor):
    """
    Вычисляет логарифмический спектр мощности.
    """
    if image_tensor.shape[1] == 3:
        gray = 0.299 * image_tensor[:, 0] + 0.587 * image_tensor[:, 1] + 0.114 * image_tensor[:, 2]
    else:
        gray = image_tensor.squeeze(1)

    fft_result = fft.fft2(gray)
    fft_shifted = fft.fftshift(fft_result)

    magnitude = torch.abs(fft_shifted)
    log_magnitude = torch.log1p(magnitude)

    mean = log_magnitude.mean(dim=[1, 2], keepdim=True)
    std = log_magnitude.std(dim=[1, 2], keepdim=True)
    log_magnitude = (log_magnitude - mean) / (std + 1e-8)
    log_magnitude = torch.clamp(log_magnitude, -1, 1)
    return log_magnitude.unsqueeze(1)

def create_bilateral_highpass_mask(H, W, pixel_sigma=0.3, freq_cutoff=0.25, device='cuda'):
    """
    Создает маску для двухстороннего фильтра высоких частот (BiHPF).
    """
    y = torch.linspace(-1, 1, H, device=device)
    x = torch.linspace(-1, 1, W, device=device)
    yy, xx = torch.meshgrid(y, x, indexing='ij')

    gaussian = torch.exp(-(xx**2 + yy**2) / (2 * pixel_sigma**2))
    pixel_mask = 1 - gaussian

    freq_radius = torch.sqrt(xx**2 + yy**2)
    freq_mask = (freq_radius >= freq_cutoff).float()

    combined = pixel_mask * freq_mask
    return combined

def apply_bilateral_highpass_filter(log_spectrum, pixel_sigma=0.3, freq_cutoff=0.25, device='cuda'):
    """
    Применяет BiHPF к логарифмическому спектру.
    """
```

```

B, C, H, W = log_spectrum.shape
device = log_spectrum.device

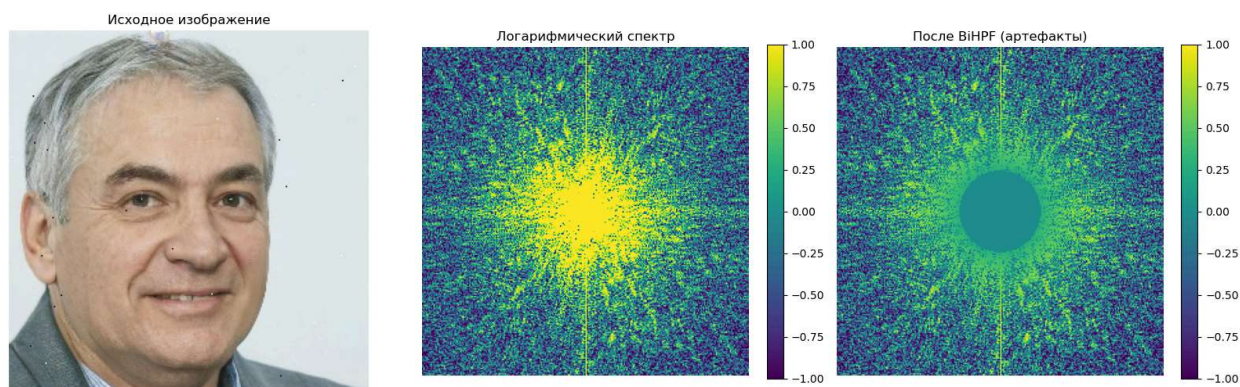
mask = create_bilateral_highpass_mask(H, W, pixel_sigma, freq_cutoff, de

filtered = log_spectrum * mask.unsqueeze(0).unsqueeze(0)
return filtered

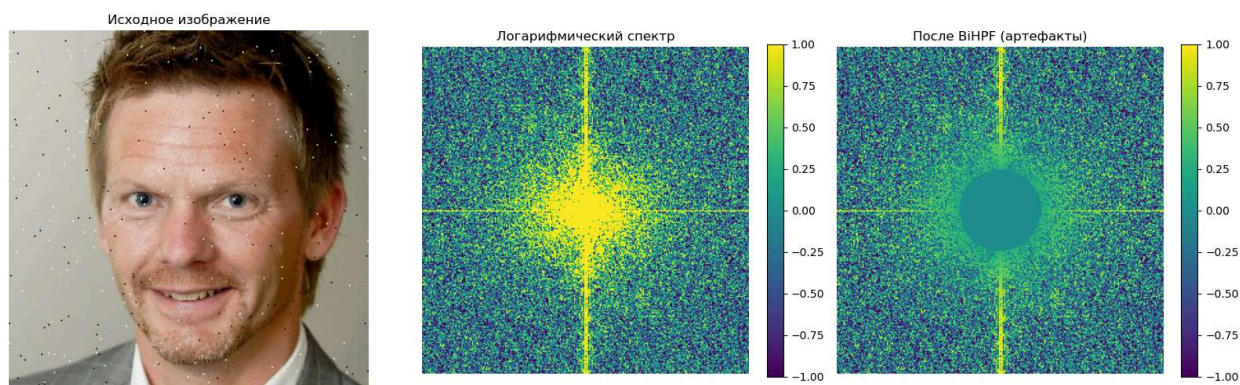
```

примеры фото, частотных карт и частотных карт после фильтра

сгенерированное

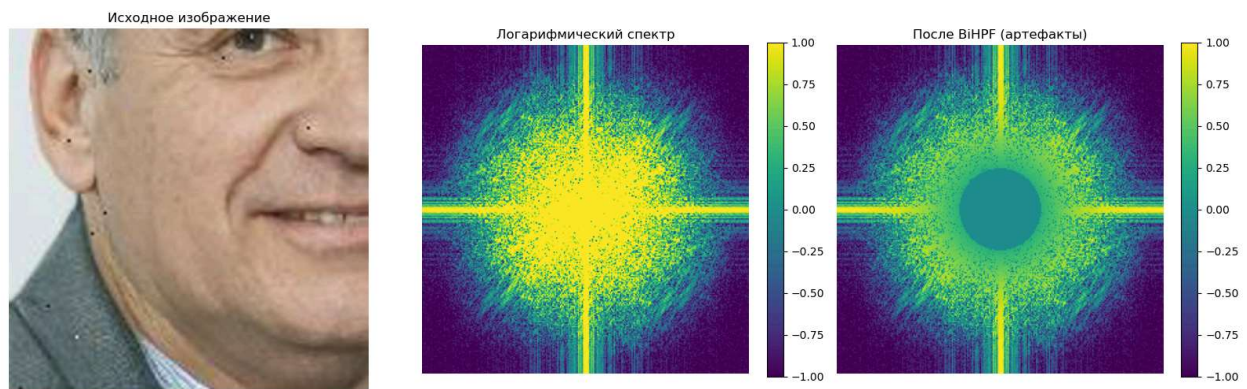


настоящее

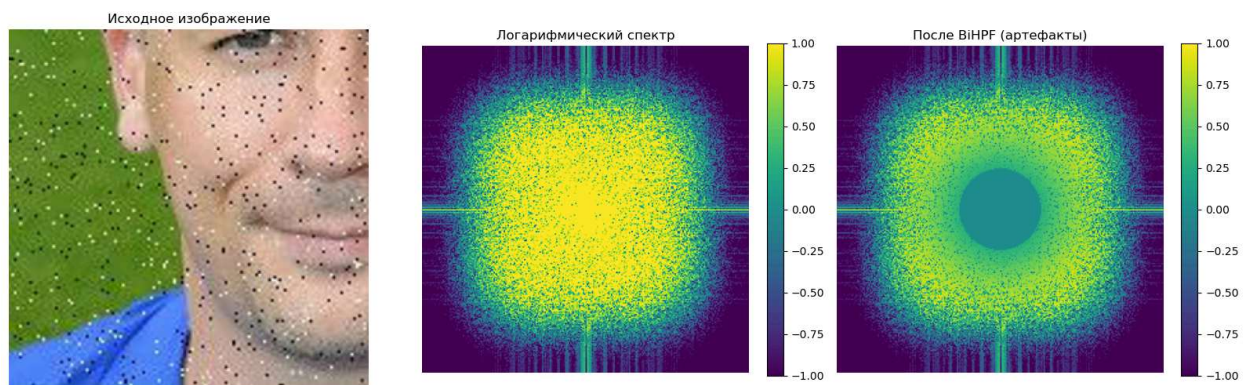


ещё лучше видна разница при приближение или рассмотрении фрагмента фото

фрагмент сгенерированного



фрагмент настоящего



наличие таких частотных артефактов объясняет бесполезность стандартных методов аугментации, так как они либо изменяют изображения, но частотная карта останется прежней, либо частотная карта будет искаженная аугментацией

сама архитектура

```
class DualDomainCNN(nn.Module):
    def __init__(self, base_channels=32):
        super().__init__()
        self.spatial_branch = nn.Sequential(
            nn.Conv2d(3, base_channels, 3, padding=1), nn.BatchNorm2d(base_channels),
            nn.Conv2d(base_channels, base_channels, 3, padding=1), nn.BatchNorm2d(base_channels),
            nn.MaxPool2d(2),

            nn.Conv2d(base_channels, base_channels*2, 3, padding=1), nn.BatchNorm2d(base_channels*2),
            nn.Conv2d(base_channels*2, base_channels*2, 3, padding=1), nn.BatchNorm2d(base_channels*2),
            nn.MaxPool2d(2),

            nn.Conv2d(base_channels*2, base_channels*4, 3, padding=1), nn.BatchNorm2d(base_channels*4),
            nn.Conv2d(base_channels*4, base_channels*4, 3, padding=1), nn.BatchNorm2d(base_channels*4),
            nn.MaxPool2d(2),

            nn.Conv2d(base_channels*4, base_channels*8, 3, padding=1), nn.BatchNorm2d(base_channels*8),
            nn.Conv2d(base_channels*8, base_channels*8, 3, padding=1), nn.BatchNorm2d(base_channels*8),
            nn.MaxPool2d(2),
```



```

        nn.Conv2d(base_channels*8, base_channels*8, 3, padding=1), nn.MaxPool2d(2),
    )

    self.freq_branch = nn.Sequential(
        nn.Conv2d(1, base_channels, 3, padding=1), nn.BatchNorm2d(base_channels),
        nn.Conv2d(base_channels, base_channels, 3, padding=1), nn.BatchNorm2d(base_channels),
        nn.MaxPool2d(2),

        nn.Conv2d(base_channels, base_channels*2, 3, padding=1), nn.BatchNorm2d(base_channels*2),
        nn.Conv2d(base_channels*2, base_channels*2, 3, padding=1), nn.BatchNorm2d(base_channels*2),
        nn.MaxPool2d(2),

        nn.Conv2d(base_channels*2, base_channels*4, 3, padding=1), nn.BatchNorm2d(base_channels*4),
        nn.Conv2d(base_channels*4, base_channels*4, 3, padding=1), nn.BatchNorm2d(base_channels*4),
        nn.MaxPool2d(2),

        nn.Conv2d(base_channels*4, base_channels*8, 3, padding=1), nn.BatchNorm2d(base_channels*8),
        nn.Conv2d(base_channels*8, base_channels*8, 3, padding=1), nn.BatchNorm2d(base_channels*8),
        nn.MaxPool2d(2),
    )

    self.fusion = nn.Sequential(
        nn.Conv2d(base_channels*16, base_channels*8, 3, padding=1),
        nn.BatchNorm2d(base_channels*8), nn.ReLU(inplace=True),
        nn.AdaptiveAvgPool2d(1),
        nn.Flatten(),
        nn.Linear(base_channels*8, 256), nn.ReLU(inplace=True), nn.Dropout(0.5),
        nn.Linear(256, 1))

    def forward(self, spatial_input, freq_input):
        spatial_feat = self.spatial_branch(spatial_input)
        freq_feat = self.freq_branch(freq_input)

        combined = torch.cat([spatial_feat, freq_feat], dim=1)

        logits = self.fusion(combined)

```

класс датасета с хранением и изображения, и спектра

```

class TestDualDataset(Dataset):
    def __init__(self, img_dir, csv_path=None, transform=None):
        self.img_dir = img_dir
        self.transform = transform

        if csv_path and os.path.exists(csv_path):
            self.df = pd.read_csv(csv_path)
            self.image_ids = self.df.iloc[:, 0].tolist()
        else:
            self.image_ids = sorted([f.split('.')[0] for f in os.listdir(img_dir)])

    def __len__(self): return len(self.image_ids)

```

```
def __getitem__(self, idx):
    img_name = self.image_ids[idx]
    img_path = os.path.join(self.img_dir, f'{str(img_name)}.jpg')
    image = Image.open(img_path).convert('RGB')

    if self.transform:
        image = self.transform(image)
    spectrum = compute_log_magnitude_spectrum(image.unsqueeze(0)).squeeze(0)
    spectrum = apply_bilateral_highpass_filter(spectrum.unsqueeze(0)).squeeze(0)

    return image, spectrum
```

✓ Сводка об обучении

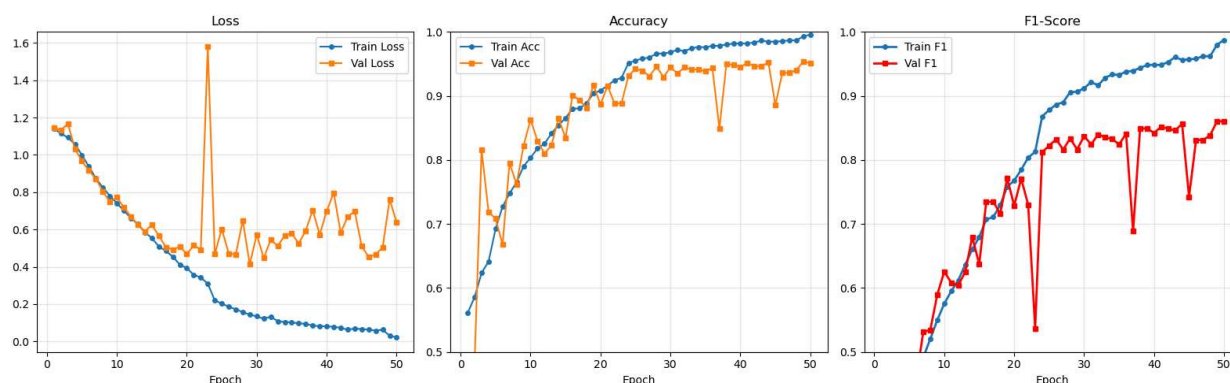
модель обучалась в 3 основных этапа

в алгоритме обучения использовались несколько способов оптимизации

- очистка памяти в начале, после обучения, после валидации
- смешанная точность вычислений (AMP)
- накопление градиента для имитации более крупных батчей

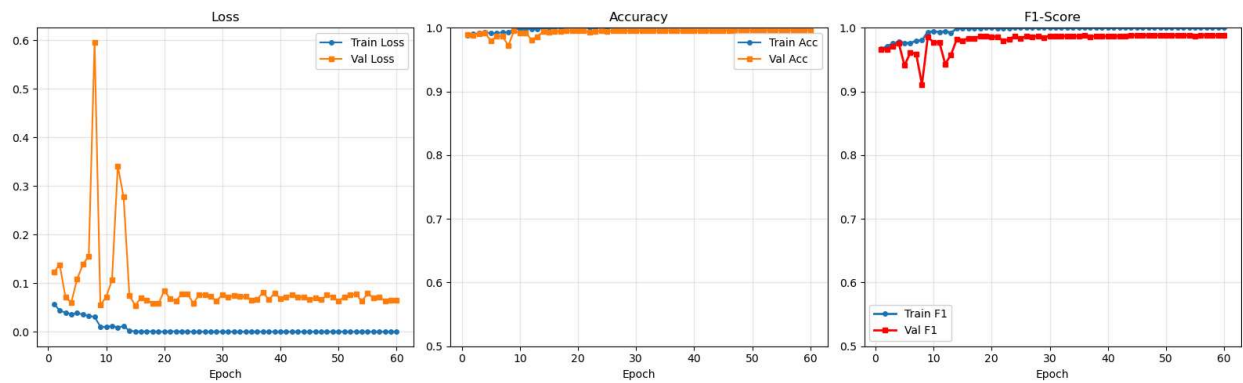
1 этап:

- эпох 50
- батч 32
- шаг накопления градиента 2



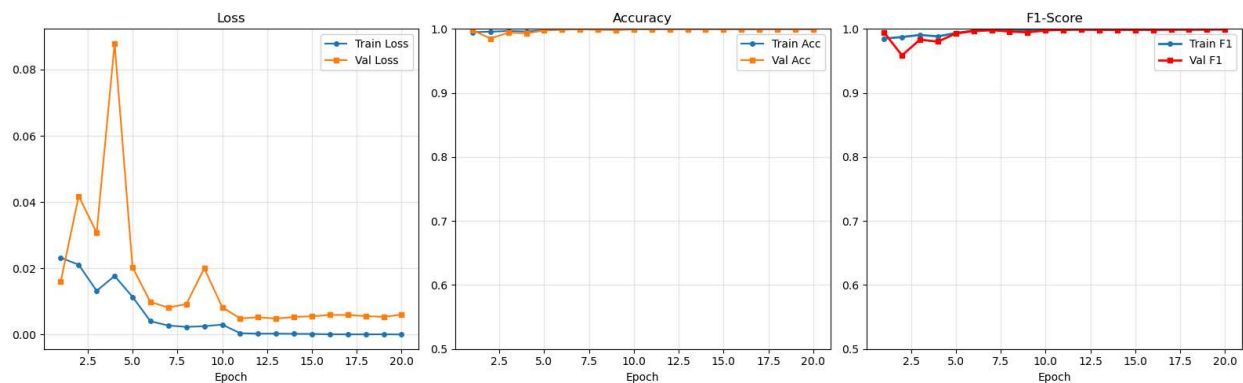
2 этап:

- эпох 60
- батч 64
- шаг накопления градиента 1



3 этап:

- эпох 20
- батч 128
- шаг накопления градиента 1



✓ Инференс и сабмит

```
model = DualDomainCNN()
model.load_state_dict(torch.load(model_path, map_location=DEVICE))
model.to(DEVICE)
model.eval()

test_transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])
])

test_dataset = TestDualDataset(test_img_path, transform=test_transform)
test_loader = DataLoader(test_dataset, batch_size=batch_size, shuffle=False)

predictions = []
image_ids = []

with torch.no_grad():
    for imgs, spec in tqdm(test_loader, desc="Inference"):
        imgs, spec = imgs.to(DEVICE), spec.to(DEVICE)
        logits = model(imgs, spec)
```

```
probs = torch.sigmoid(logits).cpu().numpy()
predictions.extend([int(p >= 0.5) for p in probs])

df_pred = pd.DataFrame({
    'id': test_dataset.image_ids,
    'target_feature': predictions})
df_pred.to_csv("submission.csv", index=False)
```

Inference: 0%| | 0/313 [00:00<?, ?it/s]

✓ Основная литература по идее

ссылки на литературу:

- <https://arxiv.org/pdf/1907.06515> (по сути основное)
- https://openaccess.thecvf.com/content_CVPR_2020/supplemental/Karras_Analyzing_and_Improving_CVPR_2020_supplemental.pdf (много математики)
- <https://arxiv.org/pdf/2109.00911> (про артефакты на высоких частотах)